

توضیح الگوریتم استفاده شده:

در این پروژه هدف این است که با استفاده از تکنیک‌های پردازش تصویر، ظاهری کارتونی به تصاویر، ویدیوها یا خروجی دوربین زنده بدهیم. این کار با ترکیب فیلترهای کلاسیک OpenCV انجام می‌شود که مرزهای تصویر را برجسته کرده و رنگ‌ها را نرم و یکدست می‌کند تا جلوه‌ای شبیه به انیمیشن‌های کارتونی ایجاد شود.

عملکرد کلی:

1. تابع اصلی: `cartoonify(image)`

این تابع با سه مرحله ساده تصویر را کارتونی می‌کند:

1. تبدیل به سطح خاکستری + بلور: برای ساده‌سازی جزئیات تصویر.
2. تشخیص لبه‌ها: با استفاده از آستانه‌گذاری تطبیقی (`adaptiveThreshold`) برای رسم خطوط برجسته.
3. تلطیف رنگ‌ها: با فیلتر `bilateralFilter` برای صاف کردن رنگ‌ها در عین حفظ لبه‌ها.
4. ترکیب نهایی: با استفاده از ماسک لبه‌ها روی تصویر تلطیف شده.

2. حالت پردازش تصویر (Image Mode):

تمام تصاویر در پوشه‌ی `images` با فرمت‌های رایج بارگذاری می‌شوند. سپس روی هر تصویر افکت کارتونی اعمال می‌شود. در نهایت تصویر اصلی و کارتونی شده در کنار هم با `matplotlib` نمایش داده می‌شوند.

3. حالت دوربین زنده (Webcam Mode):

تصویر زنده از دوربین خوانده می‌شود. سپس افکت کارتونی روی هر فریم اعمال می‌شود. همینطور نرخ فریم (FPS) روی فریم نمایش داده می‌شود و از این حلقه بینهایت با کلید `Esc` خارج می‌شود.

4. حالت ویدیویی (Video Mode):

- تمام فایل‌های ویدیویی در پوشه‌ی `videos` بارگذاری و کارتونی می‌شوند. فریم‌ها طبق نرخ 30 فریم بر ثانیه پردازش و نمایش داده می‌شوند. نرخ فریم روی تصویر درج می‌شود. از حلقه‌ی بینهایت با کلید `Esc` خارج می‌شود.

5. تابع `visualize_fps()`

Cartoonify

نرخ فریم واقعی را در هر لحظه روی تصویر درج می‌کند. توجه کنید در این تابع رنگ متن بسته به سیاه‌سفید بودن یا رنگی بودن تصویر تنظیم می‌شود.

فایل‌ها و ساختار پروژه:

```
project/
├── images/
│   ├── image1.jpg
│   └── ...
├── videos/
│   ├── image1.jpg
│   └── ...
└── app.py
```

کتابخانه‌های مورد نیاز:

برای اجرای این پروژه، کتابخانه‌های زیر باید نصب شده باشند:

```
pip install numpy matplotlib opencv-python
```

نحوه اجرا و ترتیب اجرای فایل‌ها:

تصاویر ورودی را در پوشه `images` و ویدیوهای ورودی در پوشه `videos` قرار دهید. فایل `app.py` را اجرا کنید.